Lecture 7 Study Notes: Attention and Self-Attention

CSC2516 — Neural Networks and Deep Learning

Synthesized from course lectures with worked numerical examples

Abstract

These notes develop **attention** from first principles: as an associative memory, as a soft one-nearest-neighbor classifier, and as the mechanism that fixes the information bottleneck in sequence-to-sequence models. They then specialize to **self-attention**—the core of the transformer—including path length, a fully worked numerical toy example, position encodings, and multi-headed attention. A final section shows **attention pooling** inside a small end-to-end network that maps a variable-length sequence to a scalar prediction. The notes are written as standalone study material others can reference.

Contents

1 Attention as an Associative Memory **2**

2 A Soft One-Nearest-Neighbor Classifier **3**

3 Sequence-to-Sequence with Attention **3**
- 3.1 The information bottleneck . . . 3
- 3.2 Per-step context via attention (Bahdanau et al., 2015) . . . 4
- 3.3 Where attention sits in the encoder–decoder . . . 4

4 Self-Attention Setup **4**

5 Baby Toy Example: The Ingredients **5**
- 5.1 Inputs . . . 5
- 5.2 Weight matrices . . . 6
- 5.3 Queries, keys, and values . . . 6

6 Step 1: Scaled Attention Scores **7**

7 Step 2: Softmax — "Where Should I Look?" **7**

8 Step 3: Weighted Sum of Values — "What Should I Read?" **8**
- 8.1 Intuition . . . 8
- 8.2 Numbers for the toy . . . 9

9 Putting It Together: Length T In $\rightarrow$ Length T Out **9**

10 Position Encoding: Why Self-Attention Needs Order **9**
10.1 Shared baby setup . . . 10
10.2 Absolute position embeddings (early approach) . . . 10
10.3 Method (i): Relative vector on the key . . . 11
10.4 Method (ii): Relative scalar bias . . . 12
10.5 Method (iii): RoPE (rotary position encoding) . . . 13
10.6 How position encoding preserves sequence order . . . 15

11 Why Self-Attention? Path Length **17**

12 Multi-Headed Self-Attention **17**
12.1 Intuition: what a head is (and is not) . . . 17
12.2 Definition (same notation) . . . 18
12.3 Baby example: setup . . . 18
12.4 Step A — Head 1 does ordinary self-attention . . . 19
12.5 Step B — Head 2, same $\mathbf{x}$, different weights . . . 20
12.6 Step C — Stack heads and mix with $\mathbf{W}_O$ . . . 20
12.7 Single-head vs. multi-head . . . 21
12.8 Summary of the baby walk-through . . . 22

13 Attention Pooling in a Full Network **22**
13.1 Pooling vs. self-attention . . . 22
13.2 Demonstrated example: selective marked sum . . . 22
13.3 Where attention sits in the network . . . 23
13.4 Score functions (pooling form) . . . 23
13.5 Padding and attention masks . . . 23
13.6 Training loop in brief . . . 23
13.7 Pseudocode overview . . . 24
13.8 Takeaways . . . 24

14 Cheat Sheet **24**

1 Attention as an Associative Memory

Attention is usefully viewed as the neural-network version of an *associative memory*, or key–value store. To build one, fix five ingredients:

1. a **query** — the identifier of what we want to retrieve;
2. a set of **keys** — identifiers attached to stored items;
3. a set of **values** — one payload per key;
4. a **comparison** function that scores the query against each key;
5. a **reduction** function that combines the values according to those scores.

Definition. An attention mechanism is completely specified by these five ingredients. Comparison produces a vector of scores $\mathbf{a}$; reduction maps $\{\mathbf{v}_1, \ldots, \mathbf{v}_N\}$ and $\mathbf{a}$ to a single vector in $\mathbb{R}^{d_v}$.

Python dictionary (hard attention). A dictionary compares by equality (in practice, hashing) and returns the unique matching value. That assignment is hard and non-differentiable, so it cannot sit inside a network trained by gradient descent. The rest of these notes softens the same idea: replace exact match with a continuous score, and replace hard selection with a weighted average.

Notation. Keys $\mathbf{k}_i \in \mathbb{R}^{d_k}$, query $\mathbf{q} \in \mathbb{R}^{d_q}$ (often $d_q = d_k$), values $\mathbf{v}_i \in \mathbb{R}^{d_v}$. Scores $a_i = \alpha(\mathbf{q}, \mathbf{k}_i)$. Attention acts on a *set*: shuffling key–value pairs together leaves the result unchanged (permutation invariance). That property will matter when we apply attention to sequences.
One-line intuition (kept throughout). Comparison chooses *where* to look; reduction copies *what* to read.

2 A Soft One-Nearest-Neighbor Classifier

A one-nearest-neighbor (1-NN) classifier is rarely introduced as attention, but it uses the same five ingredients and makes the soft version easy to see.

Ingredient	1-NN reading
Query	the test point $\mathbf{q}$ to classify
Keys	training inputs $\mathbf{k}_i = \mathbf{x}^{(i)}$
Values	training labels $v_i = t^{(i)}$
Comparison	negated distance $\alpha(\mathbf{q}, \mathbf{k}_i) = -\lVert\mathbf{q} - \mathbf{k}_i\rVert$
Reduction (hard)	$\text{argmax}(\mathbf{a}) \cdot V$ (label of the closest point)

Negating distance turns "closest" into "largest score," so argmax selects the nearest neighbor. Hard argmax has derivative zero almost everywhere, so it is not usable inside a trained network.

Softening. Replace argmax by softmax:

$$f(\{\mathbf{v}_1, \ldots, \mathbf{v}_N\}, \mathbf{a}) = \text{softmax}(\mathbf{a}) \cdot V.$$

The result is no longer strictly 1-NN: it averages nearby labels in proportion to closeness. Neural attention mechanisms look like this soft 1-NN, with a different comparison function and with values that need not be class labels.

> *Exam / interview note.* Framing 1-NN as attention is a common conceptual check: the five ingredients, then soft argmax via softmax.

3 Sequence-to-Sequence with Attention

Attention first became widespread in neural networks inside *sequence-to-sequence* (seq2seq) models, so we briefly revisit the encoder–decoder RNN before leaving recurrence behind.

3.1 The information bottleneck

Without attention, an encoder RNN consumes the input $x_1, \ldots, x_T$ and summarizes it with a single fixed context $\mathbf{c} = \mathbf{h}_T$ (the last encoder state). A decoder RNN is initialized from $\mathbf{c}$ and emits the output sequence. For long inputs this is a bottleneck: every relevant detail must be compressed into one fixed-length vector, and every decoder step must rely on that same summary—even when the next output word depends on only one source word.

Translation intuition. English "the cat sat on the mat" → French "le chat s'est assis sur le tapis." Producing *tapis* mainly needs *mat*, not the whole sentence. A single $\mathbf{c} = \mathbf{h}_T$ cannot specialize per output step.

3.2 Per-step context via attention (Bahdanau et al., 2015)

Compute a fresh context $\mathbf{c}_{t'}$ at each decoder step t', focused on the source positions relevant to the word about to be written. Fill in the five ingredients:

- **Query:** previous decoder state $\mathbf{s}_{t'-1}$ (cannot use $\mathbf{s}_{t'}$, which will depend on $\mathbf{c}_{t'}$);
- **Keys and values:** encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_T$;
- **Comparison:** a small MLP (encoder and decoder live in different spaces),

$$a_t = \mathbf{w}_v^\top \tanh\big(\mathbf{W}_q \mathbf{s}_{t'-1} + \mathbf{W}_k \mathbf{h}_t\big);$$

- **Reduction:** softmax-weighted average

$$\mathbf{c}_{t'} = \sum_{t=1}^{T} \alpha_t \, \mathbf{h}_t, \qquad \boldsymbol{\alpha} = \mathrm{softmax}(\mathbf{a}).$$

The decoder then consumes the context at every step,

$$\mathbf{s}_{t'} = g\big(\mathbf{s}_{t'-1},\, y_{t'-1},\, \mathbf{c}_{t'}\big),$$

e.g. by concatenating $\mathbf{c}_{t'}$ with the previous output embedding. Information about the input is no longer forced through a single vector; each output step re-selects what to read.

3.3 Where attention sits in the encoder–decoder

Figure 1 places the attention block between the encoder states (keys/values) and the decoder (query). Softmax chooses *where* among $h_1, \ldots, h_T$ to look; the weighted sum builds the context that the decoder reads.

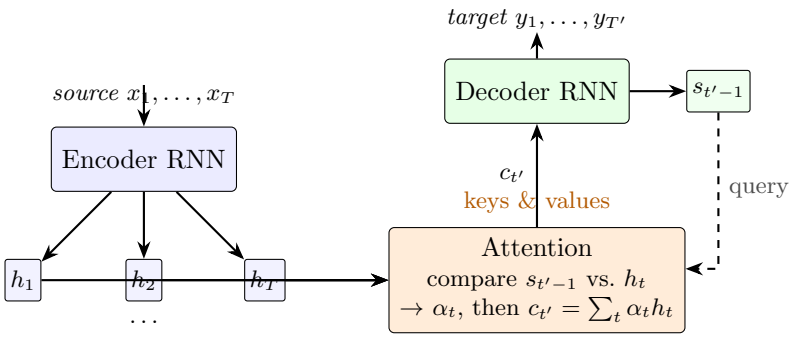
Clock convention used below: primed index t' is the *decoder / output* clock; unprimed t is the *encoder / source* clock. Encoder states stay fixed; the attention weights change with t'.

4 Self-Attention Setup

Seq2seq attention works, but the mechanism that dominates modern models is **self-attention**: queries, keys, and values all come from the *same* sequence. The layer still maps a length-T sequence to a length-T sequence (like an RNN layer), but without recurrence.

Start with token vectors $x_1, x_2, \ldots, x_T \in \mathbb{R}^m$ (e.g. word embeddings, or outputs of a previous layer). Project every token with three learned matrices shared across positions:

$$q_t = W_Q x_t, \tag{1}$$
$$k_t = W_K x_t, \tag{2}$$
$$v_t = W_V x_t. \tag{3}$$



At each decoder step t': query $= s_{t'-1}$; keys/values $= h_{1:T}$; context $c_{t'}$ is fed into the decoder together with $y_{t'-1}$.

Figure 1: RNN encoder–decoder with attention. The encoder produces states $h_1, \ldots, h_T$. At decoder step t', attention scores the previous decoder state against every encoder state, softmax-normalizes, and forms context $c_{t'}$. Without attention, a single $c = h_T$ would replace this whole block.

Vector	Role	Intuition
query q_t	"what am I looking for?"	search request
key k_t	"what do I advertise?"	index / label
value v_t	"what content do I offer?"	payload to copy

Shapes. $W_Q, W_K \in \mathbb{R}^{d_k \times m}$ and $W_V \in \mathbb{R}^{d_v \times m}$. Queries and keys share dimension d_k so a dot product is well-defined. Values may use a different d_v.

Comparison is a scaled dot product,

$$\alpha(q, k_i) = \frac{q^\top k_i}{\sqrt{d_k}},$$

and reduction is again a softmax-weighted average of the values. Scaling by $\sqrt{d_k}$ keeps score variance from growing with dimension (and prevents a saturated, nearly one-hot softmax that hurts gradients).

After the projections we have three length-T sequences—queries, keys, and values—ready to be compared. The next sections walk through every matrix with tiny numbers; afterward we return to *why* this rewrite is attractive via path length.

5 Baby Toy Example: The Ingredients

We use deliberately tiny dimensions so every matrix can be written out.

5.1 Inputs

Sequence length $T = 3$, embedding size $m = 2$:

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Think of these as embeddings of three tokens, e.g. "the", "cat", "sat". Stacked as a matrix $X \in \mathbb{R}^{T\times m}$:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

5.2 Weight matrices

Choose $d_k = d_v = 2$. Pick simple numbers (in a real model these are learned):

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}.$$

All three are 2×2 because $m = d_k = d_v = 2$.

5.3 Queries, keys, and values

For one token, e.g. x_1:

$$q_1 = W_Q x_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$
$$k_1 = W_K x_1 = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$
$$v_1 = W_V x_1 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Doing this for every token:

token	q_t	k_t	v_t
x_1	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$
x_2	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$
x_3	$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Where do v_1, v_2, v_3 come from? Each is just $v_t = W_V x_t$. Same idea as queries and keys, different matrix. The big value matrix V is those vectors stacked as rows:

$$V = \begin{bmatrix} — v_1^\top — \\ — v_2^\top — \\ — v_3^\top — \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix} \in \mathbb{R}^{T\times d_v} = \mathbb{R}^{3\times 2}.$$

Analogously one can form Q and K by stacking the $q_t^\top$ and $k_t^\top$ as rows.

6 Step 1: Scaled Attention Scores

For each query, score every key with a **scaled** dot product:

$$s_{tj} = \frac{q_t^\top k_j}{\sqrt{d_k}}.$$

Here $d_k = 2$, so $\sqrt{d_k} = \sqrt{2} \approx 1.414$.

Token 1 (q_1).

$$\begin{aligned} q_1^\top k_1 = 1 \quad &\Rightarrow \quad s_{11} = \tfrac{1}{\sqrt{2}} \approx 0.707, \\ q_1^\top k_2 = 1 \quad &\Rightarrow \quad s_{12} = \tfrac{1}{\sqrt{2}} \approx 0.707, \\ q_1^\top k_3 = 2 \quad &\Rightarrow \quad s_{13} = \tfrac{2}{\sqrt{2}} = \sqrt{2} \approx 1.414. \end{aligned}$$

Token 2 (q_2).

$$\begin{aligned} q_2^\top k_1 = 0 \quad &\Rightarrow \quad s_{21} = 0, \\ q_2^\top k_2 = 1 \quad &\Rightarrow \quad s_{22} = \tfrac{1}{\sqrt{2}} \approx 0.707, \\ q_2^\top k_3 = 1 \quad &\Rightarrow \quad s_{23} = \tfrac{1}{\sqrt{2}} \approx 0.707. \end{aligned}$$

Token 3 (q_3).

$$\begin{aligned} q_3^\top k_1 = 1 \quad &\Rightarrow \quad s_{31} = \tfrac{1}{\sqrt{2}} \approx 0.707, \\ q_3^\top k_2 = 2 \quad &\Rightarrow \quad s_{32} = \sqrt{2} \approx 1.414, \\ q_3^\top k_3 = 3 \quad &\Rightarrow \quad s_{33} = \tfrac{3}{\sqrt{2}} \approx 2.121. \end{aligned}$$

Scaled score matrix (row t = scores for query t):

$$S = \frac{QK^\top}{\sqrt{d_k}} \approx \begin{bmatrix} 0.707 & 0.707 & 1.414 \\ 0.000 & 0.707 & 0.707 \\ 0.707 & 1.414 & 2.121 \end{bmatrix}.$$

Shape: $T \times T$. Every token looks at every token.
Why divide by $\sqrt{d_k}$? Raw dot products grow with dimension. Scaling keeps the values fed into softmax from becoming extremely large, which would make the attention weights too peaked (nearly one-hot) and hard to train.

7 Step 2: Softmax — "Where Should I Look?"

Turn each row of S into weights that sum to 1:

$$A_{tj} = \frac{\exp(s_{tj})}{\sum_{j'=1}^{T} \exp(s_{tj'})}.$$

$$\alpha_1 = \text{softmax}([0.707,\ 0.707,\ 1.414]) \approx [0.248,\ 0.248,\ 0.503],$$
$$\alpha_2 = \text{softmax}([0.000,\ 0.707,\ 0.707]) \approx [0.198,\ 0.401,\ 0.401],$$
$$\alpha_3 = \text{softmax}([0.707,\ 1.414,\ 2.121]) \approx [0.140,\ 0.284,\ 0.576].$$

Attention weight matrix:

$$A \approx \begin{bmatrix} 0.248 & 0.248 & 0.503 \\ 0.198 & 0.401 & 0.401 \\ 0.140 & 0.284 & 0.576 \end{bmatrix}.$$

Each row sums to 1. Entry A_{tj} means: *how much does token t listen to token j?*

Softmax alone only gives a ranking / allocation of attention. It does not yet move any content.

8 Step 3: Weighted Sum of Values — "What Should I Read?"

This is the **retrieval** step:

$$y_t = \sum_{j=1}^{T} A_{tj}\, v_j \qquad \text{or, in matrix form,} \qquad Y = AV.$$

8.1 Intuition

- A answers *where* to look (percentages).
- V holds the *content* you might copy (after W_V).
- $y_t = \sum_j A_{tj} v_j$ builds a new vector as a blend of everyone's values, with more weight on the tokens that matched the query strongly.

Without multiplying by V, attention would only give you a ranking. With it, you get a new representation that actually contains information from the useful tokens.

Concrete reading of token 1. Weights $\alpha_1 \approx [0.25,\ 0.25,\ 0.50]$ say: "listen a little to token 1, a little to token 2, and about half to token 3." Then

$$y_1 \approx 0.25\, v_1 + 0.25\, v_2 + 0.50\, v_3$$

is mostly like v_3, with some v_1 and v_2 mixed in. Token 1's new vector is rewritten using content from the tokens its query matched.

Why this matters.

1. **Context injection.** A bare embedding of "it" is vague. After this step, "it" can absorb value-features from "cat" if its query matched the key of "cat".
2. **Soft lookup.** Hard selection would pick one token only. Softmax + weighted sum lets you grab a main token and keep a bit of others, and gradients flow through all of them.
3. **Same length, richer meaning.** You keep T outputs, but each is no longer "just me"; it is "me, plus the parts of the sequence I needed."

8.2 Numbers for the toy

$$y_1 \approx 0.248\, v_1 + 0.248\, v_2 + 0.503\, v_3 \approx \begin{bmatrix} 0.752 \\ 1.503 \end{bmatrix},$$
$$y_2 \approx 0.198\, v_1 + 0.401\, v_2 + 0.401\, v_3 \approx \begin{bmatrix} 0.599 \\ 1.401 \end{bmatrix},$$
$$y_3 \approx 0.140\, v_1 + 0.284\, v_2 + 0.576\, v_3 \approx \begin{bmatrix} 0.716 \\ 1.576 \end{bmatrix}.$$

Output matrix:

$$Y \approx \begin{bmatrix} 0.752 & 1.503 \\ 0.599 & 1.401 \\ 0.716 & 1.576 \end{bmatrix}.$$

9 Putting It Together: Length T In $\rightarrow$ Length T Out

Stage	Object	Shape
input	x_1, x_2, x_3	$T = 3$ vectors of size $m = 2$
project	Q, K, V	still T vectors each
scores	$S = QK^\top/\sqrt{d_k}$	$T \times T$
weights	$A = \mathrm{softmax}(S)$	$T \times T$
output	$y_1, y_2, y_3 = AV$	$T = 3$ vectors of size $d_v = 2$

The full self-attention formula is

$$\boxed{Y = \mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.}$$

You never collapse the sequence. Each position keeps its own output; that output is a soft mix of all values, with mix weights chosen by that position's query:

$$\underbrace{(x_1, \ldots, x_T)}_{T \text{ inputs}} \xrightarrow{W_Q, W_K, W_V} (Q, K, V) \xrightarrow{\mathrm{softmax}(QK^\top/\sqrt{d_k})\, V} \underbrace{(y_1, \ldots, y_T)}_{T \text{ outputs}}.$$

Self-attention vs. encoder–decoder attention. In the encoder–decoder case, the query came from the decoder and the states being averaged came from the encoder. In self-attention, one sequence supplies Q, K, and V, so each token can gather context from the rest of the *same* sentence (including itself).

10 Position Encoding: Why Self-Attention Needs Order

Self-attention alone is **orderless**. Attention treats keys and values as a *set*: shuffling the inputs shuffles the outputs the same way (permutation equivariance). That is fine for a bag of items, but bad for language — "the cat sat" does not mean the same as "sat the cat." To process sequences we must inject position into either the inputs or the comparison scores.

#	Method	What it does
0	Absolute embeddings (early)	Add a learned vector e_t to input x_t
(i)	Relative key shift	Add a learned vector $\mathbf{p}_{ij}$ to the key
(ii)	Relative bias	Add a learned scalar p_{ij} to the score
(iii)	RoPE	Rotate q_i and k_j by angles depending on i, j

Relative methods depend on the **offset** $i - j$ (up to some maximum), so they extrapolate past training lengths better than absolute embeddings.

10.1 Shared baby setup

Two tokens, dimension $d_k = 2$. For clarity we sometimes omit $\sqrt{d_k}$ (the story is unchanged with scaling):

$$q_1 = \begin{bmatrix}1\\0\end{bmatrix},\ q_2 = \begin{bmatrix}0\\1\end{bmatrix}, \qquad k_1 = \begin{bmatrix}1\\0\end{bmatrix},\ k_2 = \begin{bmatrix}0\\1\end{bmatrix}.$$

Without position info, scores are just $q_i^\top k_j$:

$$S = \begin{bmatrix} q_1^\top k_1 & q_1^\top k_2 \\ q_2^\top k_1 & q_2^\top k_2 \end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}.$$

If we swap the two tokens, this pattern just swaps with them. Order is not truly "seen"; only content matching is.

10.2 Absolute position embeddings (early approach)

Learn a vector e_t for each index and **add it before** projecting:

$$\tilde{x}_t = x_t + e_t, \qquad q_t = W_Q \tilde{x}_t,\ \ldots$$

Baby example. Let

$$e_1 = \begin{bmatrix}1\\0\end{bmatrix}, \qquad e_2 = \begin{bmatrix}0\\1\end{bmatrix}.$$

Suppose two tokens have the *same content* $x_1 = x_2 = \begin{bmatrix}0\\0\end{bmatrix}$ (so without PE they are identical). After adding positions,

$$\tilde{x}_1 = e_1, \qquad \tilde{x}_2 = e_2.$$

They are no longer the same, so $q_1 \neq q_2$ and $k_1 \neq k_2$. Softmax can treat "token at position 1" differently from "token at position 2" even if the words were identical.

Order preserved? Yes: if a word moves from slot 1 to slot 2, it picks up e_2 instead of e_1, so scores change.

Weakness. Embeddings are only learned up to the longest training length $T_{\max}$. A longer test sequence has no $e_{T_{\max}+1}$. This absolute approach is uncommon today for that reason.

10.3 Method (i): Relative vector on the key

Idea: leave the query alone; **add a learned position vector to the key** before the dot product. The vector depends only on the offset $i - j$:

$$\alpha(q_i, k_j) = \frac{q_i^\top (k_j + \mathbf{p}_{ij})}{\sqrt{d_k}}, \qquad \mathbf{p}_{ij} = \mathbf{p}_{i-j}.$$

Initial vectors (content only). Reuse the shared baby q and k:

$$q_1 = \begin{bmatrix}1\\0\end{bmatrix},\ q_2 = \begin{bmatrix}0\\1\end{bmatrix}, \qquad k_1 = \begin{bmatrix}1\\0\end{bmatrix},\ k_2 = \begin{bmatrix}0\\1\end{bmatrix}.$$

Content-only scores (no position yet):

$$S_{\text{content}} = \begin{bmatrix} q_1^\top k_1 & q_1^\top k_2 \\ q_2^\top k_1 & q_2^\top k_2 \end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}.$$

Position table we will add. One learned vector per offset (for $T = 2$):

$$\mathbf{p}_0 = \begin{bmatrix}0\\0\end{bmatrix}, \quad \mathbf{p}_{+1} = \begin{bmatrix}1\\0\end{bmatrix}, \quad \mathbf{p}_{-1} = \begin{bmatrix}0\\1\end{bmatrix}.$$

Step-by-step: inject position into each key, then score. For every pair (i, j):

1. Start with the content key k_j.
2. Look up the offset $i - j$ and fetch $\mathbf{p}_{i-j}$.
3. Form the position-aware key $\tilde{k}_{ij} := k_j + \mathbf{p}_{i-j}$.
4. Score: $s_{ij} = q_i^\top \tilde{k}_{ij}$ (optionally divide by $\sqrt{d_k}$).

Pair $(1, 1)$ — offset 0:

$$\begin{aligned} k_1 &= \begin{bmatrix}1\\0\end{bmatrix}, \\ \tilde{k}_{11} &= k_1 + \mathbf{p}_0 = \begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}0\\0\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix}, \\ s_{11} &= q_1^\top \tilde{k}_{11} = \begin{bmatrix}1 & 0\end{bmatrix} \begin{bmatrix}1\\0\end{bmatrix} = 1. \end{aligned}$$

Pair $(1, 2)$ — offset $1 - 2 = -1$:

$$\begin{aligned} k_2 &= \begin{bmatrix}0\\1\end{bmatrix}, \\ \tilde{k}_{12} &= k_2 + \mathbf{p}_{-1} = \begin{bmatrix}0\\1\end{bmatrix} + \begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}0\\2\end{bmatrix}, \\ s_{12} &= q_1^\top \tilde{k}_{12} = \begin{bmatrix}1 & 0\end{bmatrix} \begin{bmatrix}0\\2\end{bmatrix} = 0. \end{aligned}$$

Pair $(2,1)$ — offset $+1$:

$$k_1 = \begin{bmatrix}1\\0\end{bmatrix},$$
$$\tilde{k}_{21} = k_1 + \mathbf{p}_{+1} = \begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}2\\0\end{bmatrix},$$
$$s_{21} = q_2^\top \tilde{k}_{21} = \begin{bmatrix}0 & 1\end{bmatrix}\begin{bmatrix}2\\0\end{bmatrix} = 0.$$

Pair $(2,2)$ — offset 0:

$$\tilde{k}_{22} = k_2 + \mathbf{p}_0 = \begin{bmatrix}0\\1\end{bmatrix},$$
$$s_{22} = q_2^\top \tilde{k}_{22} = 1.$$

What you end with. Position-aware score matrix:

$$S_{(i)} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} = S_{\text{content}} \quad \text{(with this particular } \mathbf{p} \text{ table).}$$

The *keys that were actually used* are no longer the raw k_j; they are the position-shifted versions

$$\tilde{k}_{11} = \begin{bmatrix}1\\0\end{bmatrix},\ \tilde{k}_{12} = \begin{bmatrix}0\\2\end{bmatrix},\ \tilde{k}_{21} = \begin{bmatrix}2\\0\end{bmatrix},\ \tilde{k}_{22} = \begin{bmatrix}0\\1\end{bmatrix}.$$

If we change $\mathbf{p}_{+1}$ to $\begin{bmatrix}2\\0\end{bmatrix}$, then $s_{21} = q_2^\top \begin{bmatrix}3\\0\end{bmatrix} = 0$ still here, but a query with a nonzero first component would get a boost when looking *backward*. Different offsets use different $\mathbf{p}$, so "A then B" and "B then A" can produce different scores even when A and B have similar content.

Cost. One learned vector of size d_k per offset — somewhat costly.

10.4 Method (ii): Relative scalar bias

Idea: keep the content key unchanged; **add a learned scalar to the score** after the content dot product. The scalar depends only on the offset $i - j$:

$$\alpha(q_i, k_j) = \frac{q_i^\top k_j}{\sqrt{d_k}} + p_{ij}, \qquad p_{ij} = p_{i-j}.$$

Initial vectors (content only). Same baby q and k as above. Content-only score matrix:

$$S_{\text{content}} = \begin{bmatrix}q_1^\top k_1 & q_1^\top k_2\\ q_2^\top k_1 & q_2^\top k_2\end{bmatrix} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}.$$

Notice: here we do *not* modify k_j itself. The vectors q_i and k_j stay exactly as they are.

Position table we will add. One learned scalar per offset ("prefer attending to the previous token"):

$$p_0 = 0, \qquad p_{+1} = +2, \qquad p_{-1} = -1.$$

Step-by-step: compute content score, then add the bias. For every pair (i, j):

1. Start with content vectors q_i and k_j (unchanged).
2. Content score: $s_{ij}^{\text{content}} = q_i^\top k_j$.
3. Look up offset $i - j$ and fetch scalar p_{i-j}.
4. Position-aware score: $s_{ij} = s_{ij}^{\text{content}} + p_{i-j}$.

Pair $(1, 1)$ — offset 0:

$$s_{11}^{\text{content}} = 1, \quad p_0 = 0, \quad s_{11} = 1 + 0 = 1.$$

Pair $(1, 2)$ — offset -1:

$$s_{12}^{\text{content}} = 0, \quad p_{-1} = -1, \quad s_{12} = 0 + (-1) = -1.$$

Pair $(2, 1)$ — offset $+1$:

$$s_{21}^{\text{content}} = 0, \quad p_{+1} = +2, \quad s_{21} = 0 + 2 = 2.$$

Pair $(2, 2)$ — offset 0:

$$s_{22}^{\text{content}} = 1, \quad p_0 = 0, \quad s_{22} = 1 + 0 = 1.$$

What you end with.

$$S_{(ii)} = S_{\text{content}} + \underbrace{\begin{bmatrix} p_0 & p_{-1} \\ p_{+1} & p_0 \end{bmatrix}}_{\text{position bias matrix}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}.$$

Softmax on row 2 (scores $[2, 1]$) puts more weight on position 1 — the previous token — than on itself. That preference is *purely positional*; the content vectors never changed.

If you swap the sequence, the bias table is still indexed by positions (i, j), so the favored "previous" neighbor is a different word. Order matters. Cheaper than (i): one number per offset, not a d_k-vector.

10.5 Method (iii): RoPE (rotary position encoding)

Idea: do not add a table of p's. Instead, **rotate** the query by an angle depending on its absolute index i, and rotate the key by an angle depending on j. Relative position appears automatically in the dot product:

$$\alpha(q_i, k_j) = \frac{(R_i q_i)^\top (R_j k_j)}{\sqrt{d_k}}.$$

In 2D, rotation by ϕ is

$$R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}.$$

Key identity:

$$(R_i q)^\top (R_j k) = q^\top R_i^\top R_j k = q^\top R_{j-i}\, k.$$

So the score depends on the relative angle $(j - i)\theta$.

Initial vectors (content only). Take a single content direction for a clear demo:

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad k = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Without RoPE, $q^\top k = 1$ no matter where they sit. Position has not entered yet.

Position rotations we will apply. Fix $\theta = \pi/2$ (90°). Absolute rotations for indices $0, 1, 2$:

$$R_0 = R(0) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{(identity; no rotation)},$$

$$R_1 = R(\pi/2) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix},$$

$$R_2 = R(\pi) = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Step-by-step: rotate each vector by its absolute position. For a query at index i and a key at index j:

1. Start with content vectors q and k.
2. Form the position-aware query $\tilde{q}_i := R_i q$.
3. Form the position-aware key $\tilde{k}_j := R_j k$.
4. Score: $s_{ij} = \tilde{q}_i^\top \tilde{k}_j$ (equivalently $q^\top R_{j-i}\, k$).

Case A — same position ($i = j = 0$, offset 0):

$$\tilde{q}_0 = R_0 q = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$\tilde{k}_0 = R_0 k = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$s_{00} = \tilde{q}_0^\top \tilde{k}_0 = 1.$$

Initial q, k are unchanged because $R_0 = I$.

Case B — key one step ahead ($i = 0$, $j = 1$, offset $+1$):

$$\tilde{q}_0 = R_0 q = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{(query still “pointing right”)},$$

$$\tilde{k}_1 = R_1 k = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \text{(key now “pointing up”)},$$

$$s_{01} = \tilde{q}_0^\top \tilde{k}_1 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

Case C — query at 1, key at 2 (still offset $+1$):

$$\tilde{q}_1 = R_1 q = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

$$\tilde{k}_2 = R_2 k = \begin{bmatrix} -1 \\ 0 \end{bmatrix},$$

$$s_{12} = \tilde{q}_1^\top \tilde{k}_2 = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ 0 \end{bmatrix} = 0.$$

Same relative offset as Case B ⇒ same score 0, even though absolute indices differ. That is the relative property of RoPE.

What you end with.

Case	Initial q, k	After rotation	Score
$i = j = 0$	$\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}1\\0\end{bmatrix}$	$\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}1\\0\end{bmatrix}$	1
$i = 0, j = 1$	same content	$\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}$	0
$i = 1, j = 2$	same content	$\begin{bmatrix}0\\1\end{bmatrix}, \begin{bmatrix}-1\\0\end{bmatrix}$	0

Same content, different offset ⇒ different score. Relative position is baked into the geometry of the rotated vectors. RoPE needs no learned p and is common in modern LLMs.

10.6 How position encoding preserves sequence order

We close by tying the three methods back to the original problem: making self-attention care about order. Below we start from the *same* initial content vectors in two different orders and show that, once position is injected, the final attention scores (and therefore Y) differ.

Initial content vectors (no position yet). Two tokens with contents A and B:

$$x_A = \begin{bmatrix}1\\0\end{bmatrix}, \qquad x_B = \begin{bmatrix}0\\1\end{bmatrix}.$$

For a transparent demo, take $W_Q = W_K = I$, so $q = k = x$ at each position, and use method (ii) biases $p_0 = 0$, $p_{+1} = +2$, $p_{-1} = -1$.

Order 1: A then B (positions $1, 2$).

1. Place contents into slots: $x_1 = x_A = \begin{bmatrix}1\\0\end{bmatrix}$, $x_2 = x_B = \begin{bmatrix}0\\1\end{bmatrix}$.

2. Content scores:
$$S_{\text{content}} = \begin{bmatrix} x_A^\top x_A & x_A^\top x_B \\ x_B^\top x_A & x_B^\top x_B \end{bmatrix} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}.$$

3. Add position biases by offset:
$$S^{(A,B)} = \begin{bmatrix}1 & 0\\0 & 1\end{bmatrix} + \begin{bmatrix}0 & -1\\2 & 0\end{bmatrix} = \begin{bmatrix}1 & -1\\2 & 1\end{bmatrix}.$$

4. Softmax (approx.): row 2 $\text{softmax}([2, 1]) \approx [0.73, 0.27]$. So token B (position 2) pulls *mostly from A* (the previous token).

Order 2: B then A (positions $1, 2$).

1. Same contents, swapped slots: $x_1 = x_B = \begin{bmatrix}0\\1\end{bmatrix}$, $x_2 = x_A = \begin{bmatrix}1\\0\end{bmatrix}$.

2. Content scores:
$$S_{\text{content}} = \begin{bmatrix} x_B^\top x_B & x_B^\top x_A \\ x_A^\top x_B & x_A^\top x_A \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$
(Same matrix shape of 1s on the diagonal — content matching alone still cannot tell the two orders apart.)

3. Add the *same* position-bias table (indexed by slots, not by word identity):
$$S^{(B,A)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}.$$

4. Softmax again puts row 2 weight mostly on position 1 — but position 1 is now B, so token A pulls mostly from B.

What changed?

	Order A, B	Order B, A
Initial slot 1 vector	$x_A = \begin{bmatrix}1\\0\end{bmatrix}$	$x_B = \begin{bmatrix}0\\1\end{bmatrix}$
Initial slot 2 vector	$x_B = \begin{bmatrix}0\\1\end{bmatrix}$	$x_A = \begin{bmatrix}1\\0\end{bmatrix}$
After adding position	biases favor "look left"	same biases, different neighbor
Who does slot 2 read?	mostly A	mostly B

Same words, different order ⇒ different mixes in $Y = AV$. Without the position step, both orders would produce equivalent (shuffled) attention; with it, order is preserved.

Without PE vs. with PE (summary).

- **Without PE:** content alone decides scores; shuffling tokens shuffles outputs the same way; "cat sat" and "sat cat" induce the same set of interactions.
- **With PE:** position changes vectors ($x_t + e_t$, rotated q, k) or scores ($+ p_{ij}$); A_{ij} depends on both content match *and* offset/order; softmax still picks where to look and values supply what to read, but "where" is position-aware.

One-line map of the three lecture variants.

- **(i)** Start with k_j; add learned relative vector $\mathbf{p}_{i-j}$; then $q_i^\top(k_j + \mathbf{p}_{i-j})$.
- **(ii)** Start with $q_i^\top k_j$; add learned relative scalar p_{i-j} before softmax.
- **(iii)** Start with q_i, k_j; rotate to $R_i q_i$ and $R_j k_j$; their angle difference carries the offset (RoPE).

Absolute embeddings add "I am at index t" to the token itself; relative methods encode "you are $i - j$ steps away from me" inside the comparison.

11 Why Self-Attention? Path Length

Having seen self-attention compute a length-T rewrite of its input, a natural question is why this is attractive compared with convolution or recurrence. Path length measures how many sequential operations are needed to mix information between two positions of the input—closely related to the idea of a receptive field. Consider a length-5 sequence and ask when an output can depend on both x_1 and x_5.

Layer	Path length ($x_1 \leftrightarrow x_5$)	Parallelism
1D conv (kernel 3)	2 (receptive field grows by 2 per layer)	Positions in one layer are parallel; depth is s
RNN	5 (signal travels step by step)	Inherently sequential along time
Self-attention	1 (every output mixes all inputs in one layer)	All outputs computable in parallel

More generally, for length T: convolution has path length $O(T/k)$ for kernel size k; recurrence has path length $O(T)$; self-attention has path length $O(1)$. Short paths are not a guarantee that optimization will discover long-range dependencies, but they remove an architectural obstruction: information need not pass through a long chain of Jacobians.

Path length also counts work that cannot be parallelized. Convolutional layers stack depth-wise; RNNs wait on the previous time step. Self-attention outputs are independent weighted averages of the same inputs, so the layer is a pair of large matrix multiplies—well matched to modern hardware:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad Y = \mathrm{softmax}_{\mathrm{row}}(S)\, V.$$

12 Multi-Headed Self-Attention

In a transformer we do not run self-attention once. We run H **heads** in parallel — each an independent self-attention with its own weights — then aggregate them. This is analogous to multiple channels in a CNN: same input, different filters, then combine.

12.1 Intuition: what a head is (and is not)

A **head** is one full self-attention pathway: one triple of weight matrices

$$(\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)})$$

plus the scores, softmax weights, and outputs those weights induce on the shared input $\mathbf{x}$.

Structurally, every head is the same kind of object. Same shapes, same formulas, same steps (project → score → softmax → weighted values). There is *no* built-in specialty — nothing in the definition says "head 1 does syntax" or "head 2 looks at coordinate 1." The big W's are peers: different *copies* of the same type of parameters.

What multi-head actually buys you. Multi-head means: keep H separate sets of W's. That gives the model extra capacity — H chances to learn H attention patterns in parallel — then $\mathbf{W}_O$ blends them. Whether those patterns actually become usefully different is up to training (random init + the loss). If two heads ended up with identical W's, they would compute identical $\mathbf{h}^{(i)}$ and the extras would buy nothing.

Why more than one copy can help. One triple $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)$ implements only *one* soft "who matches whom" rule. Sequences often need several rules at once. Multiple parameter copies give room for that; they do not hard-code what each rule is.

One-line intuition. Multi-heads = different sets of the big W's (same architecture, separate parameters). They offer different opportunities for attention patterns; there is no inherent structural difference among the W's themselves.

12.2 Definition (same notation)

Multi-headed self-attention applies H independent self-attentions, each with its own $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)}$, and aggregates with a learned output matrix $\mathbf{W}_O$.

For each head $i = 1, \ldots, H$,

$$\mathbf{h}^{(i)} = \text{self-attention}\big(\mathbf{W}_Q^{(i)},\, \mathbf{W}_K^{(i)},\, \mathbf{W}_V^{(i)},\, \mathbf{x}\big).$$

Each $\mathbf{h}^{(i)}$ is a length-T sequence of vectors in $\mathbb{R}^{d_v}$. Stack the H head outputs at a position and mix:

$$\text{Output} = \mathbf{W}_O \begin{bmatrix} \mathbf{h}^{(1)} \\ \mathbf{h}^{(2)} \\ \vdots \\ \mathbf{h}^{(H)} \end{bmatrix}, \qquad \mathbf{W}_O \in \mathbb{R}^{d_v \times H d_v}.$$

$\mathbf{W}_O$ maps the stacked Hd_v-vector back to d_v, so the layer still emits d_v-dimensional vectors — now informed by all heads.

Flow.

$$\mathbf{x} \longrightarrow \begin{cases} \text{head } 1: \ \mathbf{W}_Q^{(1)}, \mathbf{W}_K^{(1)}, \mathbf{W}_V^{(1)} \ \to \ \mathbf{h}^{(1)} \\ \text{head } 2: \ \mathbf{W}_Q^{(2)}, \mathbf{W}_K^{(2)}, \mathbf{W}_V^{(2)} \ \to \ \mathbf{h}^{(2)} \\ \vdots \\ \text{head } H: \ \mathbf{W}_Q^{(H)}, \mathbf{W}_K^{(H)}, \mathbf{W}_V^{(H)} \ \to \ \mathbf{h}^{(H)} \end{cases} \xrightarrow{\text{stack}} \mathbb{R}^{Hd_v} \xrightarrow{\mathbf{W}_O} \mathbb{R}^{d_v}.$$

12.3 Baby example: setup

Take $T = 2$, $H = 2$, embedding size $m = 2$, and $d_k = d_v = 1$ so every projection is a single number (easy to see by hand).

Initial input vectors.

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Each $\mathbf{x}_t$ is just a length-2 list of numbers (two entries).

Head 1 weights (a teaching choice, not a definition of "head"):

$$\mathbf{W}_Q^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad \mathbf{W}_K^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad \mathbf{W}_V^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Head 2 weights (another teaching choice — different copy of W):

$$\mathbf{W}_Q^{(2)} = \begin{bmatrix} 0 & 1 \end{bmatrix}, \quad \mathbf{W}_K^{(2)} = \begin{bmatrix} 0 & 1 \end{bmatrix}, \quad \mathbf{W}_V^{(2)} = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

Why these particular numbers? Only to make the arithmetic show two heads disagreeing. With $\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \end{bmatrix}$, each projection keeps only the *top* entry of $\mathbf{x}_t$ (feature dim $2 \to 1$); sequence length stays $T = 2$. With $\mathbf{W}^{(2)} = \begin{bmatrix} 0 & 1 \end{bmatrix}$, each projection keeps only the *bottom* entry. That is a hand-picked illustration that **different** W**'s** $\Rightarrow$ **different** q, k, v **and different** A. It is *not* claiming that real heads are "the first-entry head" and "the second-entry head." In a trained model each $\mathbf{W}^{(i)}$ is a full learned map over all features.

12.4 Step A — Head 1 does ordinary self-attention

1. Project (initial $\mathbf{x} \to q, k, v$).

$$\begin{aligned}
q_1^{(1)} &= \mathbf{W}_Q^{(1)} \mathbf{x}_1 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1, \\
k_1^{(1)} &= \mathbf{W}_K^{(1)} \mathbf{x}_1 = 1, \\
v_1^{(1)} &= \mathbf{W}_V^{(1)} \mathbf{x}_1 = 1, \\
q_2^{(1)} &= \mathbf{W}_Q^{(1)} \mathbf{x}_2 = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0, \\
k_2^{(1)} &= 0, \\
v_2^{(1)} &= 0.
\end{aligned}$$

2. Score. Since $d_k = 1$, $\sqrt{d_k} = 1$, so

$$S^{(1)} = \begin{bmatrix} q_1^{(1)} k_1^{(1)} & q_1^{(1)} k_2^{(1)} \\ q_2^{(1)} k_1^{(1)} & q_2^{(1)} k_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

3. Softmax (where to look).

$$A^{(1)} = \text{row-softmax}(S^{(1)}) \approx \begin{bmatrix} 0.73 & 0.27 \\ 0.50 & 0.50 \end{bmatrix}.$$

(Row 1 favors position 1; row 2 is uniform because both scores were 0.)

4. Reduce (what to read).

$$\begin{aligned}
h_1^{(1)} &= 0.73\, v_1^{(1)} + 0.27\, v_2^{(1)} = 0.73 \cdot 1 + 0.27 \cdot 0 \approx 0.73, \\
h_2^{(1)} &= 0.50 \cdot 1 + 0.50 \cdot 0 = 0.50.
\end{aligned}$$

Head 1 output sequence:

$$\mathbf{h}^{(1)} \approx \big(0.73,\ 0.50\big).$$

Intuition. Same procedure as ordinary self-attention. Because we *chose* $\mathbf{W}^{(1)} = \begin{bmatrix}1 & 0\end{bmatrix}$, the values happen to be $(v_1^{(1)}, v_2^{(1)}) = (1, 0)$. Softmax on row 1 pulls mostly from $v_1^{(1)}$, so $h_1^{(1)}$ is large; position 2 averages, so $h_2^{(1)}$ is mid-sized. The important point is not "first entry"; it is that this particular copy of the W's induced this particular $A^{(1)}$ and $\mathbf{h}^{(1)}$.

12.5 Step B — Head 2, same x, different weights

1. Project.

$$
\begin{aligned}
q_1^{(2)} &= \mathbf{W}_Q^{(2)}\mathbf{x}_1 = \begin{bmatrix}0 & 1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} = 0,\\
k_1^{(2)} &= 0, \qquad v_1^{(2)} = 0,\\
q_2^{(2)} &= \mathbf{W}_Q^{(2)}\mathbf{x}_2 = \begin{bmatrix}0 & 1\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} = 1,\\
k_2^{(2)} &= 1, \qquad v_2^{(2)} = 1.
\end{aligned}
$$

2–4. Score, softmax, reduce.

$$
S^{(2)} = \begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}, \qquad A^{(2)} \approx \begin{bmatrix}0.50 & 0.50\\ 0.27 & 0.73\end{bmatrix}.
$$

$$
\begin{aligned}
h_1^{(2)} &= 0.50 \cdot 0 + 0.50 \cdot 1 = 0.50,\\
h_2^{(2)} &= 0.27 \cdot 0 + 0.73 \cdot 1 \approx 0.73.
\end{aligned}
$$

Head 2 output sequence:

$$
\mathbf{h}^{(2)} \approx \big(0.50,\ 0.73\big).
$$

Intuition. Step B is the *same recipe* as Step A; the only change is a *different* set of W's. With our toy $\mathbf{W}^{(2)}$, the values happen to be $(0, 1)$ and the attention pattern flips relative to head 1:

$$
A^{(1)} \approx \begin{bmatrix}0.73 & 0.27\\ 0.50 & 0.50\end{bmatrix}, \qquad A^{(2)} \approx \begin{bmatrix}0.50 & 0.50\\ 0.27 & 0.73\end{bmatrix}.
$$

Same $\mathbf{x}$; different parameter copies $\Rightarrow$ different A and $\mathbf{h}$. If instead $\mathbf{W}^{(2)} = \mathbf{W}^{(1)}$, Steps A and B would be identical — there is no magic that forces heads apart beyond having separate parameters that training may (or may not) differentiate.

12.6 Step C — Stack heads and mix with $\mathbf{W}_O$

At each position t, stack the two head outputs. Here each head output is a scalar ($d_v = 1$), so the stack is 2-dimensional:

$$
\begin{bmatrix}h_t^{(1)}\\ h_t^{(2)}\end{bmatrix} \in \mathbb{R}^{Hd_v} = \mathbb{R}^2.
$$

Lecture shape: $\mathbf{W}_O \in \mathbb{R}^{d_v \times Hd_v} = \mathbb{R}^{1\times 2}$. Pick a simple mixer that sums the two heads:

$$
\mathbf{W}_O = \begin{bmatrix}1 & 1\end{bmatrix}.
$$

Position 1 — initial stacked vector, then mix.

$$\begin{bmatrix} h_1^{(1)} \\ h_1^{(2)} \end{bmatrix} \approx \begin{bmatrix} 0.73 \\ 0.50 \end{bmatrix},$$

$$\text{output}_1 = \mathbf{W}_O \begin{bmatrix} h_1^{(1)} \\ h_1^{(2)} \end{bmatrix} \approx \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.73 \\ 0.50 \end{bmatrix} = 1.23.$$

Position 2.

$$\begin{bmatrix} h_2^{(1)} \\ h_2^{(2)} \end{bmatrix} \approx \begin{bmatrix} 0.50 \\ 0.73 \end{bmatrix},$$

$$\text{output}_2 \approx \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.50 \\ 0.73 \end{bmatrix} = 1.23.$$

What you end with. A length-$T = 2$ sequence of d_v-dimensional outputs

$$\big(\text{output}_1,\ \text{output}_2\big) \approx \big(1.23,\ 1.23\big),$$

each built from *both* heads.

Intuition for $\mathbf{W}_O$.

- $\mathbf{W}_O = \begin{bmatrix} 1 & 0 \end{bmatrix}$: keep head 1 only; ignore head 2.
- $\mathbf{W}_O = \begin{bmatrix} 0 & 1 \end{bmatrix}$: keep head 2 only.
- $\mathbf{W}_O = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}$: average the two views.
- $\mathbf{W}_O = \begin{bmatrix} 1 & 1 \end{bmatrix}$ (our choice): sum / amplify both.

In a real network $\mathbf{W}_O$ is learned, so gradient descent decides how to blend the H head outputs (whatever patterns their separate W's happened to learn) into one vector per position.

12.7 Single-head vs. multi-head

	Single-head	Multi-head
Input	$\mathbf{x}_1, \ldots, \mathbf{x}_T$	same
Weights	one $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$	H copies of the same kind of W's
Structure of each head	—	identical architecture; only parameters differ
Attention	one matrix A	H matrices $A^{(i)}$ (may or may not differ)
Before mix	d_v	Hd_v (stacked)
Final size	d_v	still d_v, via $\mathbf{W}_O$
Length	$T \to T$	$T \to T$

12.8 Summary of the baby walk-through

1. **Start:** $\mathbf{x}_1 = \begin{bmatrix}1\\0\end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix}0\\1\end{bmatrix}$.

2. **Head 1:** project with $\mathbf{W}^{(1)} \to$ scores $S^{(1)} \to$ weights $A^{(1)} \to \mathbf{h}^{(1)} \approx (0.73, 0.50)$.

3. **Head 2:** same $\mathbf{x}$, different $\mathbf{W}^{(2)} \to \mathbf{h}^{(2)} \approx (0.50, 0.73)$.

4. **Stack** at each t: $\begin{bmatrix}h_t^{(1)}\\h_t^{(2)}\end{bmatrix}$.

5. **Mix:** $\text{output}_t = \mathbf{W}_O[\cdots]$; still length T, dimension d_v.

Remember: *a head = one copy of* $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)$ *running ordinary self-attention. Multi-head* $= H$ *such copies (same structure, separate parameters), stacked and blended by* $\mathbf{W}_O$*. No inherent role is baked into any head.*

13 Attention Pooling in a Full Network

Self-attention rewrites a length-T sequence into another length-T sequence. A closely related pattern—**attention pooling**—collapses a variable-length sequence into *one* context vector, then feeds that vector to a small feed-forward head. The five attention ingredients are unchanged; only the query source and output shape differ.

13.1 Pooling vs. self-attention

	Attention pooling	Self-attention
Query	**One** learned q (or H queries for multi-head)	One $q_t = W_Q x_t$ **per position**
Keys / values	Feature vectors h_t (often after a dense layer); often $v_t = h_t$	$k_t = W_K x_t$, $v_t = W_V x_t$
Output length	**One** context c per sequence	T outputs $y_1, \ldots, y_T$
Typical role	Summarize sequence $\to$ predict a scalar / class	Rewrite each token with context

Same core reduction $c = \sum_t \alpha_t v_t$ with $\alpha = \text{softmax}(e)$; different query source and output shape.

13.2 Demonstrated example: selective marked sum

As a concrete end-to-end illustration, consider sequences whose tokens are 2D vectors (value, mark):

	Pos 0	Pos 1	Pos 2	Pos 3	...	later
value	0.5	-0.7	**0.3**	0.1	...	**0.9**
mark	0	0	**1**	0	...	**1**

- **Input** X**:** the whole sequence of these vectors.
- **Target** y**:** sum of values where mark $= 1$ (here $y = 0.3 + 0.9 = 1.2$).
- **Prediction** $\hat{y}$**:** the network's scalar output.

- **Loss:** MSE($\hat{y}$, y).

Attention receives no separate supervision. The regression loss alone trains every parameter—including the query and any scoring weights—so that α_t learns to focus on marked positions. Construction of layers creates parameters; the forward pass applies them to a batch to produce $\hat{y}$; training repeats forward → loss → backward → optimizer step.

13.3 Where attention sits in the network

$$
\begin{aligned}
&X \in \mathbb{R}^{B\times T\times 2} && \text{(batch of sequences)}\\
&h = \mathrm{Dense}_1(X) && \in \mathbb{R}^{B\times T\times d} \quad \text{(position-wise)}\\
&c = \mathrm{AttnPool}(h;\, q) && \in \mathbb{R}^{B\times d} \quad \text{(one vector per example)}\\
&\hat{y} = \mathrm{Out}(\mathrm{Dense}_2(c)) && \in \mathbb{R}^{B\times 1} \quad \text{(predicted marked sum).}
\end{aligned}
$$

Keys and values are typically both h. Softmax chooses *where* to look; h_t supplies *what* to average. This is the same reduction seen in seq2seq attention, but with a *learned* query instead of a decoder state, and with a scalar head instead of a next-token RNN.

13.4 Score functions (pooling form)

One shared query q for the whole sequence:

$$
\begin{aligned}
\text{Bahdanau (additive):} \quad & e_t = q^\top \tanh(W_a h_t + b_a),\\
\text{Scaled dot-product:} \quad & e_t = \frac{q^\top h_t}{\sqrt{d}},\\
\text{Multiplicative:} \quad & e_t = q^\top W_a h_t \quad \text{(no tanh, no bias).}
\end{aligned}
$$

Then $\alpha = \mathrm{softmax}(e)$ and $c = \sum_t \alpha_t h_t$.

Multi-head pooling. Use H queries $q^{(1)}, \ldots, q^{(H)}$ on the same keys/values, obtain H contexts, concatenate, and mix with a learned W_O back to model dimension. Same idea as multi-headed self-attention: separate parameter copies, no built-in specialty per head; the output is still one pooled vector per sequence.

13.5 Padding and attention masks

Sequences in a batch may differ in length. Padding to a common $T_{\max}$ requires a mask so padded positions do not enter the softmax:

$$
\texttt{attention_mask}[b,t] = \begin{cases} 1 & \text{if position } t \text{ is padding,}\\ 0 & \text{if it is a real token.}\end{cases}
$$

Before softmax, set $e_t = -\infty$ on pads so $\alpha_t \approx 0$. Without masking, padding steals attention mass and corrupts c.

13.6 Training loop in brief

One training step. Load (X, y, mask); forward $\hat{y} = f_\theta(X, \text{mask})$; compute $\ell = \mathrm{MSE}(\hat{y}, y)$; backpropagate (gradients reach dense layers *and* attention parameters); optimizer step (often with gradient clipping).

Validation / prediction. Same forward without gradient tracking: no parameter updates. Validation tracks generalization; prediction collects $\hat{y}$ (e.g. against y on a scatter plot) to check that the network recovers marked sums.

13.7 Pseudocode overview

```
# ----- Data -----
for each example:
    X = [(v_t, m_t)] for t = 1..T     # m_t in {0,1}
    y = sum of v_t where m_t == 1     # regression target

# ----- Batching -----
pad sequences in a batch to length T_max
mask[b,t] = 1 if pad else 0

# ----- Model forward -----
h = Dense1(X)                         # (B, T, d), position-wise
e_t = score(q, h_t)                   # Bahdanau / dot / multiplicative
e_t = -inf if mask_t else e_t
alpha = softmax(e) over t
c = sum_t alpha_t * h_t               # pooled context (B, d)
y_hat = Out(Dense2(c))                # (B, 1)

# ----- Train step -----
loss = MSE(y_hat, y)
loss.backward()
optimizer.step()
```

Multi-head variant: replace one q by $q^{(1)}, \ldots, q^{(H)}$, form H contexts, concatenate, apply W_O; still one c per example.

13.8 Takeaways

- Pooling is attention with a single (or few) queries and a length-1 output—ideal when the downstream task needs one summary, not a rewritten sequence.
- The marked-sum task shows how soft selection can discover which positions matter from a scalar loss alone.
- Masking is part of the mechanism: pads must not receive attention mass.

14 Cheat Sheet

1. **Five ingredients:** query, keys, values, comparison, reduction. Softmax chooses where; values supply what.
2. **1-NN as attention:** keys = training points, values = labels, comparison = negated distance, hard reduction = argmax; soft version uses softmax.

3. **Seq2seq + attention:** query $= s_{t'-1}$, keys/values $= h_{1:T}$, per-step context $c_{t'} = \sum_t \alpha_t h_t$ replaces the single bottleneck $c = h_T$.

4. **Self-attention:** $q_t = W_Q x_t$, $k_t = W_K x_t$, $v_t = W_V x_t$; score $s_{tj} = q_t^\top k_j / \sqrt{d_k}$; $Y = \text{softmax}(S)V$. Length T in $\rightarrow$ length T out. Path length $O(1)$; fully parallelizable.

5. **Order:** self-attention alone is orderless; absolute / relative / RoPE position encodings make scores depend on order.

6. **Multi-head:** H independent copies of (W_Q, W_K, W_V); stack head outputs; mix with W_O.

7. **Pooling:** one (or H) learned query(/ies); one context $c = \sum_t \alpha_t h_t$; feed-forward head predicts a scalar (e.g. marked sum) with MSE.

Remember: *softmax chooses where to look; values are what you read. Position encoding makes "where" depend on order, not only on content. Multi-head adds capacity via independent parameter copies, not inherently different kinds of W. Path length $O(1)$ is why self-attention mixes distant tokens in a single layer.*

References

- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*.

- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *NeurIPS*.

- Su, J., Ahmed, M., Lu, Y., et al. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*.